Mixed models in R: nlme, lme4, and glmmTMB

NRES 746

All three packages maximize a likelihood (or a restricted likelihood) with the random effects integrated out. Fit the same model in two of them and you get the same log-likelihood: the matching lines below agreed on simulated data, except where a fit was singular or a method approximate. They differ in which models they can express, how fast they are, and how they report results.

1. Which package when

- **`glmmTMB`** is the most practical default. One syntax covers Normal, Poisson, binomial, negative binomial, beta, Tweedie and other families; zero inflation; variances that depend on covariates; AR(1) and spatial structures; and crossed random effects. Ecological data often start out looking Poisson and turn out overdispersed or autocorrelated, and here that means changing one argument rather than switching packages. It's fast, especially for GLMMs. Its defaults are ML fits and Wald tests and intervals; for profile intervals use `confint(fit, method = "profile")` (variance terms come out on the log SD scale, so exponentiate them).
- **`lme4`** (`lmer()`, `glmer()`) is the better choice for inference on Normal models with few groups: `lmer()` fits by REML, `lmerTest` and `pbkrtest` give small-sample (Satterthwaite, Kenward-Roger) tests, and `confint()` profiles the likelihood by default. It's also the most widely used and documented, and its sparse-matrix methods handle very large Normal models with crossed effects. No residual correlation or unequal variances.
- **`nlme`** (`lme()`, `gls()`) still has the richest menu of residual structures for Normal data (`corCAR1()`, `corSpher()`, `varPower()`, and more), and `gls()` fits them with no random effects at all. Slower, Normal responses only, and crossed random effects are awkward.
- **Bayesian** (`brms`, Stan): priors, small numbers of groups, or models none of these can write.

	nlme	lme4	glmmTMB
Responses	Normal	Normal; glmer: GLMs	many (NB, beta, Tweedie, ...)
Crossed random effects	awkward	yes	yes
Residual correlation	correlation =	no	ar1(), ou(), exp() terms
Unequal variances	weights =	no	dispformula =
Zero inflation	no	no	ziformula =
Default fit	REML	REML (glmer: ML)	ML
Speed on large data	slow	fast	fast
Fixed-effect p-values	t tests	via lmerTest	Wald z

2. The same model in each package

Write `d` for the data frame; `site`, `plot`, and `obs` (observer) are factors.

```
# random intercept
lme(y ~ x, random = ~ 1 | site, data = d)
lmer(y ~ x + (1 | site), data = d)
glmmTMB(y ~ x + (1 | site), data = d)

# correlated random intercept and slope
lme(y ~ x, random = ~ x | site, data = d)
lmer(y ~ x + (x | site), data = d)
glmmTMB(y ~ x + (x | site), data = d)

# uncorrelated random intercept and slope
lme(y ~ x, random = list(site = pdDiag(~ x)), data = d)
lmer(y ~ x + (x || site), data = d)
glmmTMB(y ~ x + (x || site), data = d)          # or diag(x | site)

# nested: plots within sites
lme(y ~ x, random = ~ 1 | site/plot, data = d)
lmer(y ~ x + (1 | site/plot), data = d)
glmmTMB(y ~ x + (1 | site/plot), data = d)

# crossed: sites and observers (no simple lme() version)
lmer(y ~ x + (1 | site) + (1 | obs), data = d)
glmmTMB(y ~ x + (1 | site) + (1 | obs), data = d)
```

Residual correlation and unequal variances (not available in `lme4`). In `glmmTMB`, correlation enters as a random-effect term with one level per observation; `dispformula = ~0` removes the separate residual variance so the term *is* the residual, matching `nlme`.

```
# AR(1) within each plot, no random effects
gls(y ~ x, correlation = corAR1(form = ~ year | plot), data = d)
glmmTMB(y ~ x + ar1(yearf + 0 | plot), dispformula = ~0, data = d)    # yearf <- factor(year)

# random site intercept plus AR(1) within each plot
lme(y ~ x, random = ~ 1 | site, correlation = corAR1(form = ~ year | site/plot), data = d)
glmmTMB(y ~ x + (1 | site) + ar1(yearf + 0 | plot), dispformula = ~0, data = d)

# AR(1) with missing years (glmmTMB's ar1() treats observed years as consecutive)
gls(y ~ x, correlation = corAR1(form = ~ year | plot), data = d)      # handles gaps
glmmTMB(y ~ x + ou(tt + 0 | plot), dispformula = ~0, data = d)       # tt <- numFactor(year)

# exponential spatial correlation within each site
gls(y ~ x, correlation = corExp(form = ~ east + north | site), data = d)
glmmTMB(y ~ x + exp(pos + 0 | site), dispformula = ~0, data = d)     # pos <- numFactor(east, north)

# a separate residual variance for each treatment
gls(y ~ x, weights = varIdent(form = ~ 1 | trt), data = d)
glmmTMB(y ~ x, dispformula = ~ trt, data = d)
```

Other `nlme` options: `corCAR1()` (continuous time), `corCompSymm()`, `corGaus()`, `corSpher()`; `varPower(form = ~ fitted(.))`, `varExp(form = ~ x)`. Other `glmmTMB` options: `cs()` (compound symmetry), `gau()`, `mat()` (Matern), `us()` and `diag()` for random-effect covariances. See `?corClasses`, `?varClasses`, and `vignette("covstruct", package = "glmmTMB")`.

Non-Normal responses (`nlme` has none).

```
# Poisson counts
glmer(count ~ x + (1 | site), family = poisson, data = d)
glmmTMB(count ~ x + (1 | site), family = poisson, data = d)

# successes out of n trials
glmer(cbind(s, n - s) ~ x + (1 | site), family = binomial, data = d)
glmmTMB(cbind(s, n - s) ~ x + (1 | site), family = binomial, data = d)

# negative binomial (variance = mu + mu^2 / theta)
glmer.nb(count ~ x + (1 | site), data = d)                    # slower, approximate
glmmTMB(count ~ x + (1 | site), family = nbinom2, data = d)   # nbinom1: variance = mu * (1 + phi)

# zero-inflated negative binomial (constant probability of an extra zero)
glmmTMB(count ~ x + (1 | site), family = nbinom2, ziformula = ~ 1, data = d)

# others: beta_family() for proportions in (0, 1), tweedie(), Gamma(link = "log"), ...
```

Add an offset for effort or area with `+ offset(log(area))` in the formula, in any package.

3. Fitting options

	nlme	lme4	glmmTMB
REML/ML (default 1st)	method = "REML" / "ML"	REML = TRUE / FALSE (lmer)	REML = FALSE / TRUE
GLMM integration	--	Laplace; nAGQ = (one effect)	Laplace
Spot a bad fit	--	isSingular(fit)	diagnose(fit)

Convergence warnings? Try another optimizer:

```
lme(..., control = lmeControl(opt = "optim"))
lmer(..., control = lmerControl(optimizer = "bobyqa"))
allFit(fit)                                  # refit an lme4 model with every optimizer and compare
glmmTMB(..., control = glmmTMBControl(optimizer = optim, optArgs = list(method = "BFGS")))
```

- REML gives less biased variance estimates. Use ML when comparing models with different fixed effects (by AIC or a likelihood ratio test). Note that `lmer()` and `glmmTMB()` have different defaults, so their default fits of the same Normal model differ slightly.
- `anova(fit1, fit2)` gives a likelihood ratio test in all three. Testing a variance at zero puts it on a boundary, so that p-value is conservative (too large).
- A **singular fit** (a variance estimated at zero, or a correlation at ± 1) means the data can't support that random-effect structure. Simplify it (drop the correlation, or the random slope) rather than trusting the estimates. Different packages may stop at slightly different points on such a flat surface.
- Center and rescale predictors before blaming the optimizer.

4. Getting results out

	`nlme`	`lme4`	`glmmTMB`
Fixed effects	`fixef(fit)`	`fixef(fit)`	`fixef(fit)$cond`
Group deviations	`ranef(fit)`	`ranef(fit)`	`ranef(fit)`
Group coefficients	`coef(fit)`	`coef(fit)`	`coef(fit)`
Variance components	`VarCorr(fit)`	`VarCorr(fit)`	`VarCorr(fit)`
Confidence intervals	`intervals(fit)`	`confint(fit) (profile)`	`confint(fit) (Wald)`
Mean predictions	`predict(fit, level = 0)`	`predict(fit, re.form = NA)`	`predict(fit, re.form = NA)`
GLMM response scale	--	`type = "response"`	`type = "response"`
Simulate new data	--	`simulate(fit)`	`simulate(fit)`
Residual checks	`ACF(fit, resType = "n")`	`DHARMa (below)`	`DHARMa (below)`

- `summary()`, `logLik()`, `AIC()`, and `nobs()` work everywhere. Log-likelihoods are comparable across packages for the same data and the same estimation method (ML vs. REML).
- `glmmTMB` keeps separate parts for the mean (`$cond`), zero inflation (`$zi`), and dispersion (`$disp`).
- For GLMMs, ordinary residual plots are hard to read. The `DHARMa` package simulates from the fitted model and gives residuals that should be uniform if the model is right.
- `broom.mixed::tidy(fit)` returns a data frame of estimates from any of the three.

5. Things that trip people up

- **Names for the same thing.** The AR(1) coefficient is `Phi` in `nlme` and appears as a correlation labeled `(ar1)` in `glmmTMB`'s `VarCorr()`. With AR(1) errors, both report the SD of the total error, $\sigma/\sqrt{1-\rho^2}$, not the innovation SD.
- **Time and space in `glmmTMB`.** `ar1()` needs a factor of equally spaced times and treats the *observed* times as consecutive, so a missing year becomes a lag of 1. Use `ou()` with `numFactor()` times when there are gaps. In `nlme`, `corAR1()` needs integer times and handles gaps correctly; use `corCAR1()` for non-integer times.
- **Unique times.** Each `time | group` combination must appear once. Two observations in the same plot and year need an extra grouping level.
- **Factors.** Make grouping variables factors. `glmmTMB` time terms need a factor (`ar1()`) or `numFactor()` (`ou()`, `exp()`).
- **Missing values** stop `nlme` fits; add `na.action = na.omit`.
- **p-values.** `lmer()` reports none by design (the degrees of freedom are unclear). `lmerTest` adds approximate ones; with few groups, treat any of them as rough.
- **Few groups.** With fewer than about five levels, a random-effect variance is poorly estimated in every package. Consider fixed effects or a Bayesian fit with an informative prior.

Learn more: Bolker's GLMM FAQ, https://bbolker.github.io/mixedmodels-misc/glmmFAQ.html; Bates et al. (2015, *J. Stat. Software* 67(1)) for `lme4`; Brooks et al. (2017, *R Journal* 9: 378-400) and the `glmmTMB` vignettes; Pinheiro and Bates (2000), *Mixed-Effects Models in S and S-PLUS*; Zuur et al. (2009), *Mixed Effects Models and Extensions in Ecology with R.*